Mitigating Ciphertext Side-Channel Attacks against TEE-Shielded Neural Networks

Qianhui Dai
Shanghai Jiao Tong University
Shanghai, China
daiqh@sjtu.edu.cn

Zhen Huang
Shanghai Jiao Tong University
Shanghai, China
xmhuangzhen@sjtu.edu.cn

Guoxing Chen[✉*]
Shanghai Jiao Tong University
Shanghai, China
guoxingchen@sjtu.edu.cn

Yan Meng
Shanghai Jiao Tong University
Shanghai, China
yan_meng@sjtu.edu.cn

Yinqian Zhang
Southern University of Science and Technology
Shenzhen, China
yinqianz@acm.org

Haojin Zhu
Shanghai Jiao Tong University
Shanghai, China
zhu-hj@sjtu.edu.cn

Abstract

As Neural Network (NN) inference is increasingly deployed in Trusted Execution Environments (TEEs) to protect model intellectual property and user privacy, the issue of side-channel leakage has attracted growing attention. Ciphertext side channels constitute a representative and practically important category of model functionality leakage and user privacy breach in protected NN inference, as exemplified by HyperTheft and CipherSteal. Meanwhile, existing defenses are mainly designed for cryptographic software and use taint analysis to locate and protect sensitive writes. These taint-based solutions could incur high performance overhead for NN inference, as all memory writes will be tainted as sensitive given that the entire computation flow involves model weights and user inputs.

This paper aims to design a more efficient mitigation scheme to secure NN inference. We first present a preliminary study analyzing the actual contribution of leaked ciphertext side-channel information to the success of existing attacks. We then formally model the ciphertext side-channel leakage inherent in NN inference. Guided by this formulation, we propose a mitigation scheme that localizes the computation and memory locations most critical to feature leakage. By perturbing these specific points, we suppress both local and downstream leakage while preserving the functional behavior of the NN as much as possible. Experimental results show that the proposed defense reduces semantic inference from ciphertext side channels across multiple datasets, while preserving task utility with only moderate end-to-end overhead.

CCS Concepts

• **Security and privacy → Side-channel analysis and countermeasures**.

Keywords

Trusted execution environment, Ciphertext side channel

ACM Reference Format:
Qianhui Dai, Zhen Huang, Guoxing Chen, Yan Meng, Yinqian Zhang, and Haojin Zhu. 2026. Mitigating Ciphertext Side-Channel Attacks against TEE-Shielded Neural Networks. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830454.3846817

1 Introduction

Confidential computing has emerged as a compelling deployment model for NN training and inference in the cloud, addressing the dual requirements of protecting proprietary models from exposure and ensuring the confidentiality of user inputs and intermediate activations [8, 10, 20, 25, 39, 43, 46, 64]. Such protection is enabled by TEEs, which leverage hardware-based isolation and memory encryption to guarantee the integrity and confidentiality of code and data, even against a compromised host operating system or hypervisor. Major implementations include AMD SEV [35], SEV-ES [33], and SEV-SNP [34], Intel SGX [29, 32] and TDX [30], and Arm CCA [6]. TEEs that instantiate confidential Virtual Machines (VMs) achieve workload isolation from a malicious cloud operator by transparently encrypting the virtual machine's memory [5, 47, 55]. Unfortunately, recent work has shown that deterministic memory encryption, a practical design choice for high-performance random access at scale, can itself become an attack surface [42, 61, 62]. Specifically, an adversary with ciphertext visibility can correlate whether and how ciphertext blocks change at fixed addresses with the victim's computation, even when the implementation is conventionally constant-time.

This vulnerability is especially pronounced for deep learning workloads. Modern NN execution is characterized by large-scale tensor write operations, such as activations, partial sums, normalization statistics and attention scores. Moreover, numerous operators repeatedly write to the same memory regions across iterations and layers [7, 26, 58], which can be used to analyze the possibility of ciphertext collisions. Recent exploits such as CipherSteal [62] and HyperTheft [61] have demonstrated that ciphertext side channels can be exploited to recover sensitive information from TEE-shielded NN, including user inputs and model functionalities.

[*]Guoxing Chen is the corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '26, The Hague, Netherlands
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
https://doi.org/10.1145/3830454.3846817

A natural defense direction is to remove deterministic ciphertext patterns using generic software hardening. Systems such as CipherFix [57] and ZebraFix [48] use taint analysis or comprehensive function annotation to identify writes that may depend on secrets, and then protect those writes with masking, freshness, or related transformations. These approaches are attractive because they aim for broad coverage, but they can be too expensive for NN inference. In an NN workload, taint or functional dependency graph propagation from the private input and protected weights quickly propagates through activations, partial sums, and operator outputs. Such conservative protection policies can therefore mark nearly every write as sensitive, greatly increasing the memory traffic. As a result, a defense that is appropriate for general software or cryptographic routines may not be the right performance-security tradeoff for NN inference.

This motivates a more workload-aware question: before protecting every secret-dependent write, what does the ciphertext trace actually leak in NN attacks? Our analysis of CipherSteal and HyperTheft (Sec. 3) shows that a substantial part of the apparent attack quality comes from the attacker's offline prior rather than from the victim trace alone. CipherSteal combines trace-derived evidence with a generative image prior, so realistic textures and fine visual details in the reconstructed image may be prior-filled rather than directly leaked. HyperTheft similarly conditions an offline-trained generator or hypernetwork using the trace; the generated surrogate model can inherit much of its structure from the attacker's prior over tasks.Thus, visual plausibility or functional plausibility does not by itself prove that the ciphertext trace revealed all recovered details.

Based on this observation, we introduce a prior-aware model for ciphertext side-channel risk in NN inference (Sec. 4). The model separates what an attacker can produce from prior knowledge alone from the additional victim-specific utility provided by the real trace. This distinction is important for defense design: the goal is not to prevent an attacker from sampling plausible images or plausible surrogate models from a strong offline prior, but to reduce the trace-specific evidence that aligns those outputs with the victim execution. At the execution level, we identify two dominant forms of leakage evidence in the traces: static-value collisions, where the packed value belongs to a set of semantically distinguished values, and transition collisions, where consecutive writes leave the packed memory state unchanged. ReLU zeros and zero-gated accumulation updates are common sources of these collisions in NN inference.

We then design a selective perturbation-based defense around these leakage sources (Sec. 5). Instead of protecting all tainted writes, we perturb only locations that are likely to create high-value collision evidence. For example, a small input perturbation reduces early transition collisions in the first layer, while ReLU zero-lifting replaces selected exact zeros with very small positive codewords. This breaks repeated-zero static collisions and also reduces downstream transition collisions caused by zero activations. The perturbation scale is chosen to survive the relevant packed arithmetic while keeping model accuracy nearly unchanged.

Our evaluation shows that this targeted strategy provides a better tradeoff for NN inference than generic all-write hardening (Sec. 7). With an appropriate perturbation scale, model utility is essentially preserved while static-value and transition collisions are substantially reduced. End-to-end evaluation against CipherSteal-style reconstruction further shows that the defended traces no longer provide enough useful collision evidence for the attack to succeed. At the same time, because the defense modifies only a small set of collision-prone operations, it incurs much lower overhead than applying CipherFix- or ZebraFix-style protection.

The main contributions of this paper are as follows.

- We revisit CipherSteal and HyperTheft, showing that their apparent recovery quality combines trace-derived evidence with substantial offline prior knowledge.
- We formulate a prior-aware model for ciphertext side-channel risk that separates prior-only attack capability from the additional victim-specific utility contributed by the real trace.
- We propose a selective perturbation-based defense instead of instead of protecting all contaminated or functionally constrained writes.
- We evaluate collision reduction, end-to-end attack effectiveness, model utility and performance, showing that the proposed defense substantially reduces leakage while preserving accuracy and incurring lower overhead than generic CipherFix- or ZebraFix-style instrumentation.

2 Background

2.1 NN Dataflow and Memory Materialization

An NN inference is a composition of parameterized tensor operators. Let $h_0 = X$ be the input and $h_\ell = f_\ell(h_{\ell-1}; W_\ell)$, $\ell = 1, \ldots, L$, where W_ℓ is the parameter set of the ℓ-th operator, h_ℓ is its intermediate result, and $Y = h_L$ is the final output. Each intermediate state therefore inherits data dependence from the input and from the sequence of model parameters applied before it.

In deployed systems, this dataflow is implemented by compiler/runtime stacks that perform graph rewriting, operator fusion, layout transformation, scheduling, and memory planning before dispatching backend kernels [9, 14, 40, 51]. These transformations connect the abstract tensor graph to concrete memory events. Because runtimes reuse buffers to reduce memory footprint, the same physical address can be written repeatedly by different operators or by different stages of the same operator [9, 51]. The resulting memory-write trace is thus an execution-level record of how intermediate values are materialized and propagated during inference.

2.2 TEEs and Ciphertext Side Channels

VM-based confidential computing platforms, including AMD SEV, SEV-ES, and SEV-SNP, protect guest memory from privileged software by encrypting private pages outside the CPU trust boundary [33–35]. The hypervisor cannot directly read guest plaintext, but it can observe the encrypted memory image and its evolution over time [34, 35]. SEV-SNP additionally enforces integrity for private pages, so stale or inconsistent ciphertext is rejected by the guest [34]. These mechanisms prevent direct plaintext disclosure and many integrity attacks, but they do not hide ciphertext values from a privileged observer [1, 2].

TEE memory encryption must support random access over large memories, so it is typically modeled as deterministic block encryption with an address-dependent tweak rather than per-write freshness. In an XOR-Encrypt-XOR(XEX)-style construction,

$$C = E_k(P \oplus T(a)) \oplus T(a), \tag{1}$$

where P is the plaintext block, a is the physical address, and $T(a)$ is the address-derived tweak. For a fixed address, the tweak is fixed; therefore, repeated writes of the same plaintext block produce repeated ciphertext blocks [19, 35]. Without freshness, writing the same plaintext block to the same physical address produces the same ciphertext. A privileged observer can therefore detect ciphertext collisions and repeated transitions at fixed addresses, even though the plaintext remains encrypted.

Prior work has shown that this leakage is practical. CIPHERLEAKS recovered RSA and ECDSA private keys from AMD SEV ciphertext observations even for constant-time OpenSSL implementations [42]; later work showed that leakage also arises from other guest memory regions such as stacks and heaps [18, 41]. Recent attacks extended ciphertext side channels to TEE-shielded NN inference [61, 62]. Sec. 8 provides a more detailed related work of ciphertext side-channel attacks, analysis tools, and defenses.

3 Revisiting CipherSteal and HyperTheft

Ciphertext side-channel attacks against TEE-shielded NN inference have been demonstrated to produce compelling attack outputs, including visually plausible input image reconstruction (*e.g.*, CipherSteal), or functionally equivalent surrogate models (*e.g.*, HyperTheft). Note that besides online observed ciphertext traces, the demonstrated attacks also rely on offline prior during the reconstruction. This motivates us to revisit these attacks to figure out:

- (Sec. 3.1) Where does the information used to reconstruct unseen input details or functionalities come from?
- (Sec. 3.2) Can these unseen details compromise victim privacy?
- (Sec. 3.3) What are security implications of these unseen details?

3.1 Attribution of Unseen Details

To begin with, we take the perspective of information theory to analyze the information flows of CipherSteal and HyperTheft. To reduce translation overhead for readers familiar with these attacks, we use each paper's native notation in the attack-specific discussion.

CipherSteal Reconstruction Flow. CipherSteal explicitly decomposes input image reconstruction into two stages. First, a transformation network $\mathcal{T}$ maps the ciphertext trace c collected from an execution on private input $x \in \mathbb{X}$ into an input-aligned sketch $h = \mathcal{T}(c)$ where h may resemble a blurred or partially recovered image. Second, a reconstruction stage uses an offline-trained image prior to produce a realistic reconstruction:

$$x^* = G(z^*), \quad z^* \in \arg\min_z L(h, I_\omega(G(z))) \tag{2}$$

where G denotes a generative model that maps a point $z \in \mathbb{Z}$ from a continuous latent space $\mathbb{Z}$ to an image, and I_ω denotes an inversion of the reconstruction that tries to produce the corresponding sketch h' from $G(z)$, with the goal of minimizing the differences between h' and h.

This pipeline is powerful because the offline image prior provides the natural image structure missing in the original trace, making the restored result visually convincing. However, this supplementation of prior knowledge actually clarifies the scope of the contribution range of the victim's ciphertext side-channel leakage. Conditioned on the offline attack components $K_{\text{CS}} = (\mathcal{T}, G, I_\omega)$ used in CipherSteal, the attack process can be viewed as a Markov chain [12, 13]:

$$x \longrightarrow c \longrightarrow h \longrightarrow x^* \tag{3}$$

By the data processing inequality [50], post-processing h with a generative prior cannot create additional victim-specific information about x beyond what is already contained in h. Using the mutual information, we have:

$$I(x; x^* \mid K_{\text{CS}}) \le I(x; h \mid K_{\text{CS}}) \le I(x; c \mid K_{\text{CS}}) \tag{4}$$

Thus x^* may contain more pixels, sharper texture, and more recognizable objects than h, but those additional details are not necessarily trace-leaked details. CipherSteal's offline components are trained on an auxiliary dataset $\mathbb{X}'$ that is disjoint from the victim private input $x \in \mathbb{X}$, that is, $\mathbb{X}' \cap \mathbb{X} = \emptyset$. Accordingly, details that appear only after the reconstruction step should be interpreted as *unseen, prior-filled details*: they are supplied by K_{CS} and the learned image distribution, not as private instance details copied from the victim input through the ciphertext trace.

HyperTheft Reconstruction Flow. HyperTheft has an analogous structure, but the target is model weights or functionality rather than the input. In its online stage, a ciphertext collision trace s from one victim execution is fed to an encoder E, producing a low-dimensional condition (a latent variable) $z = E(s)$. A decoder or hypernetwork D then generates surrogate weights $\widehat{\mathcal{W}} = D(z) = D(E(s))$ which induces a surrogate DNN $\widehat{F}$. The offline stage trains E and D on attacker-owned tasks, models, and traces so that $D \circ E$ learns how to map trace-derived conditions to functional surrogate weights.

As presented by the authors, the decoded surrogate model may exhibit functionality for classification tasks not present in prior knowledge, *i.e.*, *unseen functionalities* [61]. While this reflects the generalization ability learned by the hypernetwork D, it does not imply that the decoder introduced new victim-specific information during online recovery. Since D is trained using datasets within the same task domain of the real victim, the recovery capability can be attributed to the attacker's offline prior knowledge as a learned functional manifold.

Here, z, produced by the encoder with the ciphertext side-channel trace s as input, plays a role similar to CipherSteal's sketch h: it is the part of the victim execution that the real trace manages to condition. The high-dimensional surrogate weight vector $\widehat{\mathcal{W}}$ is not copied from the trace. It is generated by a hypernetwork prior that has learned correlations among weights and tasks. Conditioned on HyperTheft's offline attack components $K_{\text{HT}} = (E, D, \Phi)$, where Φ denotes the attacker's auxiliary task family, the victim-specific path is

$$F \longrightarrow s \longrightarrow z \longrightarrow \widehat{\mathcal{W}} \longrightarrow \widehat{F} \tag{5}$$

Consequently, the generated surrogate cannot contain more victim-specific functionality information than the trace-derived condition

z provides:

$$I(F;\widehat{F} \mid K_{\mathrm{HT}}) \le I(F;z \mid K_{\mathrm{HT}}) \le I(F;s \mid K_{\mathrm{HT}}). \quad (6)$$

In other words, K_{HT} encodes extensive prior information about the task and functional domain. If z can only weakly identify the victim's target function, the hypernetwork may still generate a seemingly reasonable classifier with high confidence, corresponding to a priori supported but misaligned target. Mapping the ciphertext trace to a classification boundary does not necessarily mean learning the actual boundaries or functions performed by the victim. Without additional information such as victim queries and real labels, an attacker might not be able to detect this misalignment.

Takeaway 1: From the perspective of information theory, the attacker's offline prior cannot provide more information about the victim inputs or model weights beyond the leaked ciphertext traces.

3.2 Effect of Unseen Details

While previous mutual information analysis showed that the final reconstruction could not provide more victim-specific information than what was already preserved in the trace-derived latent representation, attacks like CipherSteal do take extra effort to generate additional unseen details. Next, we explore how these additional details contributes to infer victim privacy.

CipherSteal Predictive Performance. Since ciphertext traces reflect the feature extraction process related to the victim model F's inference tasks, the inference results represent the most likely channel for private information leakage.

We evaluate the predictive performance of the reconstructed images (the sketch h and the visually plausible x^*) by comparing them against to the original victim image x. Each is fed into the victim model F to obtain the respective predictions y_h, y_{x^*}, and y_x.

Furthermore, to determine whether reconstructing an image (or even a sketch) is a necessary prerequisite for better prediction leakage, we trained a multi-layer perceptron (MLP) model F_c to perform direct inference from raw ciphertext traces. For each victim image x, the raw ciphertext trace generated during its processing by the victim model F is fed into the trace-based inference model F_c to obtain a prediction y_c.

Letting y_g denote the ground-truth label, we measure:

- **Prediction Consistency:**, Measures the fidelity of the attack by determining if the reconstructed (or inferred) results match the victim's original output (*i.e.*, $y_h = y_x$, $y_{x^*} = y_x$, or $y_c = y_x$).
- **Prediction Accuracy:** Assesses the objective correctness of the results by checking if the predictions match the ground-truth label (*i.e.*, $y_h = y_g$, $y_{x^*} = y_g$, or $y_c = y_g$).

The results are shown in Table 1. As shown across multiple datasets, the visually plausible reconstruction x^* consistently performs worse than the sketch h in terms of prediction consistency. For example, on the MNIST dataset, consistency decreases from 0.8620 to 0.6960, while for the chest X-ray dataset, it drops from 0.6266 to 0.5439. Regarding prediction accuracy, the visually plausible reconstruction x^* also consistently performs worse than the sketch h. For instance, accuracy drops from 0.5340 to 0.4040 on the CIFAR dataset, and from 0.5015 to 0.3430 on the ImageNet dataset. These results indicate that the unseen details introduced during the refinement process serve as generative hallucinations rather than accurate feature reconstructions.

Table 1: CipherSteal predictive performance.

Dataset	Accuracy				Consistency		
	$y_x = y_g$	$y_h = y_g$	$y_{x^*} = y_g$	$y_c = y_g$	$y_h = y_x$	$y_{x^*} = y_x$	$y_c = y_x$
CIFAR	0.7860	0.5340	0.4040	0.7721	0.6120	0.4240	0.8827
ImageNet	0.8350	0.5015	0.3430	0.7744	0.5095	0.3435	0.8128
MNIST	0.9820	0.8620	0.6880	0.9740	0.8620	0.6960	0.9700
Chest X-ray	0.7768	0.6647	0.5317	0.7328	0.6266	0.5439	0.7124

The results related to y_c further demonstrate that the direct inference model F_c consistently outperforms h-based predictions in both prediction consistency and accuracy. Across all datasets, the y_c results are closer to the reference labels y_x than those of y_h, achieving consistency scores ranging from 0.7124 to 0.9700, and thus demonstrating the effectiveness of inference recovery from the trace alone. These results indicate that while the sketch h provides a visually interpretable representation of the input, it serves as a lossy intermediate for prediction leakage.

Takeaway 2: The unseen details are more likely to be generative hallucinations than leaked features, incurring a performance penalty when attempting to recover the victim's actual inference results.

3.3 Security Implications of Unseen Details

While the preceding quantitative analysis leverages ground-truth labels and actual inference results to demonstrate that unseen details incur a performance penalty, such data is unavailable in a real-world online attack. As assumed in CipherSteal and HyperTheft, the adversary observes only the ciphertext traces. Consequently, the attacker possesses only reconstructed images or surrogate models without any independent metrics to verify whether these outputs accurately reflect the victim's private input or the target model's behavior. We next analyze how this lack of confidence metrics leads to systematic overestimation of attack efficacy by both attackers and defenders.

Analysis of Incorrect Reconstructions. We first examine instances where the predictions derived from reconstructed images diverge from the victim's results in CipherSteal. As illustrated in Fig. 1(a), an improvement in visual realism does not necessarily mean higher fidelity of the victim's input information. In categories **I** and **II**, where x^* fails to maintain the same prediction as x, the generator still produces a visually plausible image that can be misidentified as a different category (*e.g.*, a completely different digit in the MNIST dataset).

Even when prediction consistency is maintained, the refined reconstructions often harbor significant semantic inaccuracies. In categories **III** and **IV**, where x^* maintains a consistent prediction as x, synthetic *unseen details* could deviate from the ground truth (*e.g.*, an airplane of a different model facing the opposite direction). Such hallucinations are highly misleading if the adversary intends

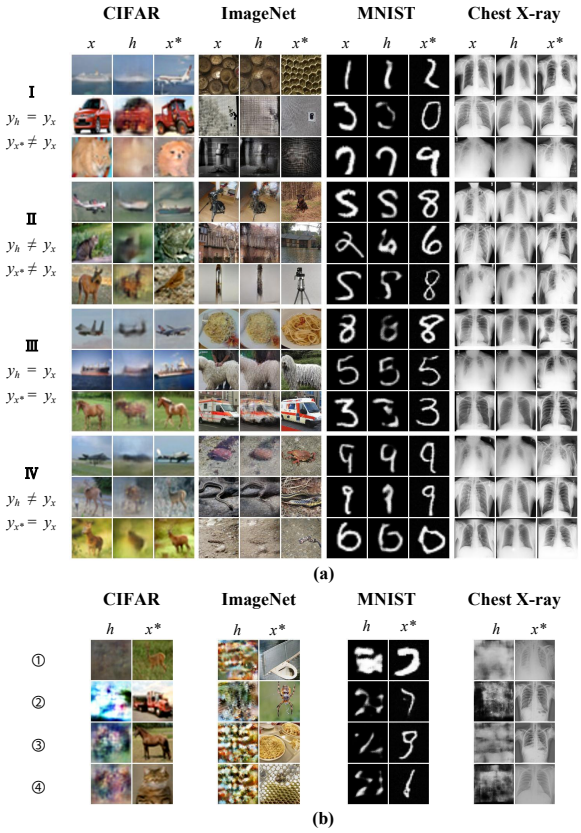


Figure 1: Characterization of generative hallucinations: semantic misclassification (Cats. I–II) and feature-level inaccuracies (Cats. III–IV) in Fig. 1(a), and generative hallucinations from collision-free traces (①) and randomized collisions (②③④) in Fig. 1(b).

to recover fine-grained details beyond the prediction label (a task for which the direct inference model F_c proves more effective).

Next, we examine HyperTheft's generalization capabilities for more unseen pairs. Since the encoder and decoder are optimized using binary classifiers derived from the attacker's prior dataset, the resulting surrogate models likely inherit their discriminative capabilities from these priors. To evaluate this, we compare the generalization performance of the original prior classifiers against the generated surrogate models. Specifically, we sampled unseen pairs and calculated their respective classification accuracies. As illustrated in Fig. 2, all classifiers exhibit comparable performance on unseen tasks; notably, the peak accuracy is consistently achieved by one of the prior classifiers rather than the surrogates. This suggests that while the generative process yields functional surrogate models, the attacker's prior knowledge serves as the primary source of discriminative power for unseen inputs. In this context, the ciphertext trace does not provide a functional advantage but merely serves as a selection mechanism for a pre-existing capability, reinforcing the conclusion that the attack's apparent utility is largely an inheritance from the attacker's prior.

Furthermore, for unseen pairs where all prior classifiers achieve low accuracy, the generated surrogate models similarly fail to distinguish the inputs. This indicates that the generative process cannot synthesize new discriminative features beyond those present in the attacker's prior knowledge; if the underlying prior is insufficient to characterize a specific pair, the resulting surrogate model remains equally incapable of effective classification.

Reconstruction under Idealized Defenses. Next, we explore two *perfect defense* scenarios to test the limits of generative attacks: (1) Collision-free traces, where all execution-level collisions are eliminated, and (2) Randomized collisions, where trace patterns occur stochastically. As shown in Fig. 1(b), CipherSteal continues to produce visually plausible images even when meaningful leakage is suppressed via collision-free traces (①) and randomized collisions (②③④). This occurs because the generative prior, having been trained on the attacker's own data, is capable of synthesizing realistic structural features from noise.

Likewise, as illustrated in Fig. 2, HyperTheft remains capable of generating surrogate models that appear to distinguish specific unseen pairs, provided that the underlying prior classifiers possess that discriminative capability. These results confirm that both visual and functional plausibility can be maintained even when the ciphertext trace carries no exploitable, victim-specific information. This reinforcing evidence suggests that the perceived success of an attack may be an artifact of the attacker's pre-existing knowledge base rather than a measure of actual data leakage.

Security Implications. This distinction is critical for robust defense design. If attack success is evaluated primarily through the lens of visual realism, generative hallucinations may be misidentified as actual ciphertext leakage. Lacking ground-truth access to the victim input, the attacker cannot discern whether recovered details are authentic or merely plausible. Similarly, the defender may overestimate the severity of the threat based on the attacker's claims and incorrectly conclude that the ciphertext trace alone reveals substantially more information than it actually contains, even though this additional information may originate from misleading information generated by the prior. Such overestimations can drive the implementation of unnecessarily broad and costly defenses (*e.g.*, universal write-back hardening, pervasive masking, or hardware-level freshness).

> **Takeaway 3:** Misattributing unseen details to actual information leakage leads both attackers and defenders to overestimate threat levels, driving inefficient, high-overhead defenses against non-exploitable generative hallucinations.

Remark. Consequently, these attacks carry an inherent interpretive risk: the online trace provides only a partial condition, while the offline prior fills in the missing structure. A visually realistic reconstruction is not, by itself, evidence that the trace carried the recovered information. On the other hand, this attribution limit does not make the ciphertext trace harmless: as Table 1 shows, direct inference from the raw trace achieves high prediction accuracy ($y_c = y_g$) and consistency ($y_c = y_x$) with the victim's prediction. This leakage, distinct from the plausible details supplied by the

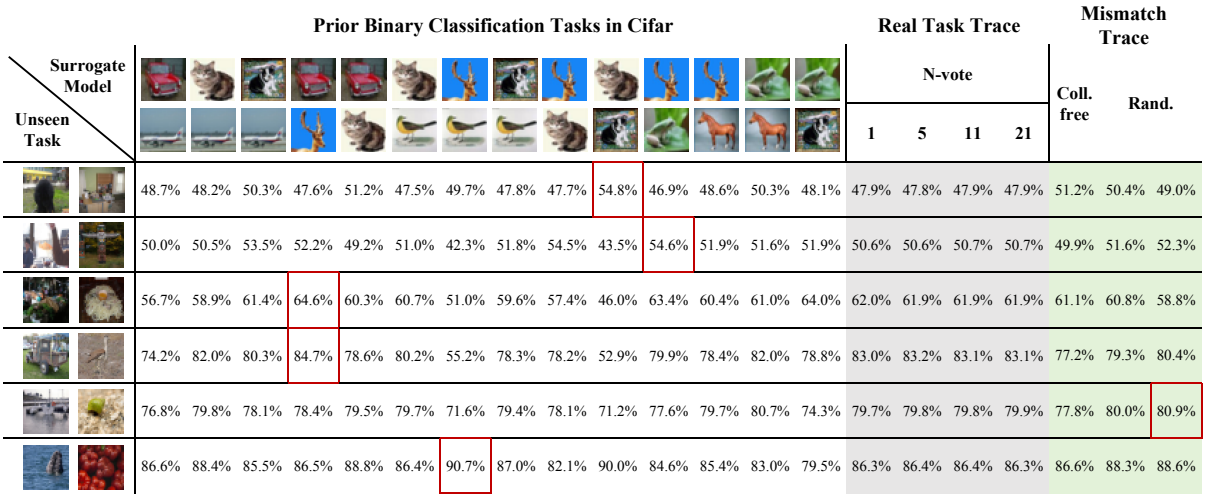


Figure 2: Generalization Capabilities of Prior Classifiers (White) vs. Surrogate Models Trained on Real (Gray) and *Perfectly Defended* Ciphertext Traces (Green). Peak accuracies are highlighted with red borders.

prior, warrants a dedicated countermeasure. This motivates a prior-aware formulation that disentangles three distinct quantities: the attacker's prior capability, the information contributed by the real ciphertext trace, and the residual ambiguity remaining after observation. We therefore evaluate attacks across prior-only, trace-assisted, and mismatched-trace configurations. From this perspective, the defense goal is not to prevent an attacker from sampling plausible images from a strong prior; rather, it is to suppress the victim-specific marginal improvement provided by the trace by weakening execution-level equalities and silent-update patterns.

4 Threat Model and Problem Formulation

Building on the distinction above between prior-generated plausibility and victim-specific trace leakage, this section aims to formally unify the attack capabilities and scope of this scenario, including the attacker's observations under deterministic memory encryption, and the role of attacker-generated priors in reconstruction attacks such as CipherSteal and HyperTheft.

4.1 Threat Model

We assume a privileged adversary that can read encrypted guest memory, record ciphertext blocks and obtain ciphertext traces over time together with the system physical address, but cannot directly recover the plaintext contents. This observation capability may arise at the software level through privileged host access or at the hardware level through physical access to the target machine. The adversary cannot compromise the trusted processor or violate the integrity guarantees.

The utility of these ciphertext observations stems from a crucial property of guest memory encryption: it is deterministic at a fixed physical address, up to the address-derived tweak used by the memory-encryption engine. We present the following formal description. Let a be a fixed physical address, $M_{a,t}$ the packed plaintext block written to that address at time t, and $C_{a,t}$ the corresponding ciphertext. Then

$$C_{a,t} = C_{a,t'} \iff M_{a,t} = M_{a,t'}. \tag{7}$$

With such deterministic encryption, repeated ciphertext observations over time can manifest as patterns such as equality and frequency distributions, which may be obtained, for example, through hypervisor access to encrypted guest memory combined with SEV-SNP page relocation [52, 60], or through DRAM-bus monitoring of encrypted memory transactions [11].

On the other hand, we consider integrity-breaking attacks beyond the scope of our paper. For example, BatteringRAM manipulates DRAM address lines to create runtime aliases, allowing stale ciphertexts to replace current protected memory and be accepted as legitimate state by SEV-SNP [15]. Mitigating such attacks requires hardware-enforced memory integrity and freshness protection.

Besides, following the adversary capabilities of prior ciphertext side-channel attacks against NN [61, 62], we also assume that the adversary can align observations to relevant execution windows using repeated queries, controlled-channel capabilities, or coarse scheduling control, and knows the network architecture, the inference code, and the placement of operators of interest.

4.2 Prior-Aware Attack Model

Confidential NN inference is a joint computation over the private input and model parameters. Let $S = (X, W)$ denote the protected state, where X is the private input and W is the victim model's weights. The ciphertext trace is not generated by X or W in isolation. It is generated by intermediate states of the joint computation $C = \mathrm{Exec}(X, W)$, including packed activations, partial sums, update states, and operator outputs.

We separate what the trace can directly constrain from what the attacker wants to recover. The trace-proximal execution evidence is $Z_{\mathrm{exec}} = \psi_{\mathrm{exec}}(C)$, which captures equality patterns, silent updates, and packed-state repetitions. The attack target is a projection of the same secret execution: $T_\tau = \pi_\tau(X, W)$ under objective τ. And

O is the ciphertext trace available to the adversary, containing the complete raw ciphertext records. The relationship between the physical evidence and the attack target is

$$T_\tau \leftarrow (X, W) \rightarrow Z_{\text{exec}} \rightarrow O. \tag{8}$$

For CipherSteal-style attacks, $T_X = X$ or an input-derived representation. For HyperTheft-style attacks, $T_W = W$ or $T_F = F$, where F denotes the functionality induced by the weights.

The attacker also has prior knowledge K, such as public architecture information, auxiliary data, and offline-trained generators or hypernetworks. This prior exists before observing the victim trace and may fill in image details, weight structure, or plausible functionality. We therefore credit the ciphertext side channel only with the improvement obtained from the real trace beyond this prior.

Let A denote a concrete attack or evaluation procedure. With only the prior, it outputs an estimate $A(K)$ of the target; with the real trace, it outputs $A(K, O)$. Let $d_\tau(\cdot, \cdot)$ be a target-dependent distortion. For a fixed attack A, the prior-only and trace-assisted risks are

$$\mathcal{R}^\tau_{\text{prior}}(A, K) \triangleq \mathbb{E}[d_\tau(T_\tau, A(K)) \mid K], \tag{9}$$

$$\mathcal{R}^\tau_{\text{trace}}(A, K) \triangleq \mathbb{E}[d_\tau(T_\tau, A(K, O)) \mid K], \tag{10}$$

and measure the incremental victim-specific information from the ciphertext trace by

$$\Delta^\tau_{\text{SC}}(A, K) \triangleq \mathcal{R}^\tau_{\text{prior}}(A, K) - \mathcal{R}^\tau_{\text{trace}}(A, K). \tag{11}$$

When the attack pipeline is fixed by the experiment, we omit A from the notation. A worst-case version can be obtained by minimizing the two risks over a chosen class of attacks, but the concrete-attacker form is the quantity used in evaluation.

Input-Reconstruction Instantiation. For CipherSteal-style reconstruction, $T_X = X$ or an input-derived representation. The trace constrains Z_{exec}, and the attacker maps that evidence back to an input using its image or data-distribution prior. Good visual reconstructions may therefore contain details supplied by the prior.

Functionality-Recovery Instantiation. For HyperTheft-style recovery, let $\mathcal{F}$ denote the functionality space under consideration, e.g., the family of input-output behaviors evaluated on the task distribution. The victim weights W induce a functionality $F \in \mathcal{F}$, and surrogate weights $\widehat{W}$ induce $\widehat{F} \in \mathcal{F}$. The attack succeeds when $d_{\mathcal{F}}\left(F, \widehat{F}\right) \leq \kappa_{\mathcal{F}}$. Here, $d_{\mathcal{F}}$ may be instantiated as prediction disagreement, loss gap or regression error on a task distribution, and $\kappa_{\mathcal{F}}$ is the evaluation tolerance for counting the surrogate as functionally close to the victim. This is the same model as input reconstruction: O provides execution evidence from the joint computation, while the prior maps that evidence toward a weight or functionality target.

Let $\mathcal{F}_K \subseteq \mathcal{F}$ denote the functionality family supported by the attacker's offline prior K, i.e., functionalities that are plausible under the attacker's auxiliary tasks, public architecture knowledge, and learned generators or hypernetworks. This prior support exists before observing the victim trace; it should not be interpreted as information directly contained in s or O. Let $\mathcal{F}_{\text{train}} \subseteq \mathcal{F}$ denote the functionalities used to train the attacker's offline generator, and let $\mathcal{F}_{\text{heldout}} \subseteq \mathcal{F}$ denote victim functionalities not used in that offline training. This gives a precise interpretation of unseen functionalities:

$$F \in \mathcal{F}_{\text{heldout}}, \quad \mathcal{F}_{\text{heldout}} \cap \mathcal{F}_{\text{train}} = \varnothing, \quad F \in \mathcal{F}_K \subseteq \mathcal{F}. \tag{12}$$

Here, *unseen* means held out from $\mathcal{F}_{\text{train}}$, not outside the attacker's prior support. A held-out victim functionality can still lie in the broader prior-supported family $\mathcal{F}_K$ because K generalizes from the offline training functionalities. If a prior-only or mismatched-trace attacker can already generate functional weights, that capability is prior-driven. The ciphertext side-channel contribution is only the additional improvement obtained when the real victim trace is used.

Practical Evaluation Considerations. For practical evaluation, it is also useful to compare the real trace against a mismatched trace $O^\perp$ drawn independently of the current victim execution but from the same workload family. This yields the counterfactual trace utility

$$\text{TU}_\tau(A, K) \triangleq \mathbb{E}\left[d_\tau(T_\tau, A(K, O^\perp)) - d_\tau(T_\tau, A(K, O)) \middle| K\right]. \tag{13}$$

Alternatively, beyond the distortion definition d_τ, with an evaluation metric that is naturally reported as a positively oriented recovery score s_τ, the same trace utility can then be equivalently written as

$$\text{TU}_\tau(A, K) = \mathbb{E}\left[s_\tau(T_\tau, A(K, O)) - s_\tau(T_\tau, A(K, O^\perp)) \middle| K\right]. \tag{14}$$

Under either representation, a positive value means that the real trace leads to better recovery than the mismatched trace.

Having quantified the victim-specific contribution of the trace to a particular attack pipeline using $\Delta^\tau_{\text{SC}}(A, K)$ and $\text{TU}_\tau(A, K)$, we next characterize the underlying execution evidence.

4.3 Packed-State Evidence in Ciphertext Traces

Let $\text{pack}(\cdot)$ denote the deterministic representation of a value or block actually written to memory after the implementation's layout, datatype conversion, rounding, padding, and block packing. Thus $\text{pack}(Y_{a,t}) = \text{pack}(Y_{a,t-1})$ means that the memory image at the monitored address is unchanged, even if the mathematical update was executed.

At a fixed physical address, Eq. (7) gives an exact correspondence between packed-plaintext equality and ciphertext equality. Therefore, for brevity, we use these packed states for the subsequent analysis.

Static-value Collisions. For a packed block $M_{a,t} \triangleq \text{pack}(Y_{a,t})$, we call a packed value *static* if it is a distinguished value induced by the computation (*e.g.* zero is a static value at a ReLU output), that is always associated with a large point mass in the packed-value distribution. Let $p_a(m) \triangleq \Pr[M_{a,t} = m]$. The static-value collision probability is defined as:

$$\text{CP}_a \triangleq \Pr[M_{a,t} \in \mathcal{S}_a] = \sum_{m \in \mathcal{S}_a} p_a(m), \tag{15}$$

where $\mathcal{S}_a$ is the semantically induced set of these static values.

Transition Collisions. A transition collision occurs when consecutive writes to the same address carry the same packed value:

$$\text{TCP}_a \triangleq \Pr[M_{a,t} = M_{a,t-1}]. \tag{16}$$

For update-style computations $Y_{a,t} = Y_{a,t-1} + \Delta_{a,t}$, the corresponding collision condition is $\text{pack}(Y_{a,t-1} + \Delta_{a,t}) = \text{pack}(Y_{a,t-1})$. This condition measures how often an update produces a transition collision: the update may be executed, but the packed value, and therefore the ciphertext at that address, remains unchanged.

Why ReLU Zeros Are a Dominant Leakage Source. We adopt the attack-surface perspective of CipherSteal and HyperTheft. These attacks show that NN operators do not contribute equally to ciphertext side-channel leakage. Operators that repeatedly produce the same encoded values expose stronger equality patterns to the adversary [61, 62]. Under this perspective, activation functions are natural leakage sites because they can transform widely distributed intermediate values into smaller and more structured output domains. ReLU is the most extreme case because its computation, defined as $y = \text{ReLU}(x) = \max(x, 0)$, maps all negative inputs to the same output value of zero. Let $p_0 \triangleq \Pr[\text{pack}(Y_{a,t}) = 0]$ denote the probability of this zero-valued packed representation. Accordingly, the static-value collision probability of packed ReLU outputs contains a dominant zero term:

$$\text{CP}_a^{\text{ReLU}} = p_0 + (\text{nonzero static-value collision terms}). \tag{17}$$

Upstream ReLU zeros (denoted by $Y_{u,t-1} = 0$) also propagate into downstream accumulations, *i.e.*, $\Delta_{a,t} = w_{a,t} Y_{u,t-1} = 0$. We have $\text{pack}(Y_{a,t-1} + \Delta_{a,t}) = \text{pack}(Y_{a,t-1})$. Hence the transition collision probability of a packed ReLU downstream outputs also contains a dominant zero term:

$$\text{TCP}_a \geq p_0. \tag{18}$$

Eqs. (17) and (18) show that ReLU-induced sparsity amplifies leakage in two ways simultaneously: it produces repeated zero outputs directly and also creates deterministic no-change events in downstream update-style operators.

Practical Evaluation Considerations. For practical evaluation, we record a trace of security-sensitive memory writes over a finite ensemble of executions. From this trace, we quantify the static-value and transition collisions, with their respective counts denoted as N_{static} and N_{trans}. To account for variations in trace length, we normalize both collision counts by the total number L of writes in the recorded trace:

$$\rho_\bullet \triangleq \frac{N_\bullet}{L}, \quad \bullet \in \{\text{static}, \text{trans}\}, \tag{19}$$

which measure the collision evidence associated with Z_{exec} as manifested in the finite trace.

5 Design

This section details our proposed defense mechanism. We first establish a formal abstraction for the defense, identifying the specific program objects targeted for transformation and the security objectives those transformations must satisfy (Sec. 5.1). We then present our concrete realization (Sec. 5.2). Finally, we analyze the scheme's security, demonstrating how this unified mechanism mitigates the two leakage phenomena formalized previously (Sec. 5.3).

5.1 Formal Defense Abstraction

Defense Objective. The primary objective of the proposed defense against information leakage in NNs is not to prevent attackers from generating seemingly plausible samples or surrogate models based on their offline prior information K. Such prior-driven behavior can still remain possible even when the ciphertext trace provides little victim-specific information. Instead, our objective is to reduce the additional alignment supplied by the real trace O over prior-only or mismatched-trace baselines $O_\perp$.

Formally, following the notation established in Sec. 4.2 and Sec. 4.3, depending on whether a concrete attack is under consideration or not, the objectives of a defense scheme D are:

- When a concrete attack A is under consideration, the objective is to limit the ciphertext side-channel information gain $\Delta^{\tau}_{\text{SC}}(A, K)$ and $\text{TU}_\tau(A, K)$ of A to a small extent.
- When no concrete attacks are specified, the objective is to reduce the collision evidence in the execution level, *i.e.*, ρ_{static} and ρ_{trans}.

Beside mitigating attacks, a practical defense should incur minimal application-level utility loss of victim NN (*e.g.*, below ϵ) and performance overhead (*e.g.*, the additional inference-time overhead below B.)

Protected Points and Defense Surface Let $\mathcal{M}$ denote the set of all memory writes during inference. The defense method then comprises two components: the protected set of writes $\mathcal{M}_\mathcal{P} \subseteq \mathcal{M}$ and the protection configuration $\theta \in \Theta(\mathcal{M}_\mathcal{P})$, which specifies how the selected writes are protected, for example, through `freshness`, `masking`, or other state randomization primitives. So, the family of defense schemes can be written as:

$$\left\{ D_{\mathcal{M}_\mathcal{P}, \theta} : \mathcal{M}_\mathcal{P} \subseteq \mathcal{M},\ \theta \in \Theta(\mathcal{M}_\mathcal{P}) \right\}. \tag{20}$$

Defenses such as Cipherfix and Zebrafix are contamination-oriented and identify memory writes that may carry sensitive values by using taint analysis or comprehensive function annotation:

$$\mathcal{M}_{\mathcal{P},\text{Sem}} = \{a \in \mathcal{M} : \text{SemanticScope}(a) = 1\}. \tag{21}$$

These approaches are suitable for broad software coverage. However, in NN inference, the semantic scope from private inputs X and protected weights W propagates rapidly to most activations, partial sums, and operator outputs. Such coverage effectively pushes freshness or masking into the entire inference path, leading to a lot of unnecessary over-defense. Following our defense objective, let $\widehat{\mathcal{M}_\mathcal{P}}$ denote the set of protected points. In particular, we prioritize points that contribute substantially to:

- First-layer input-originated no-change transitions;
- Static repeated-zero collisions at ReLU outputs;
- Downstream zero-gated transition collisions in accumulation-style updates;
- Value-preserving data movement of intermediate tensors, such as tensor copies.

This selection aligns the implementation with the execution latent Z_{exec}.

5.2 Concrete Realization

Numeric Tolerance Makes Value Rewriting Practical. In NN inference, bounded perturbations to intermediate tensors can often be tolerated when confined to low-significance bits [22]. This enables a design point different from masking or store interleaving: instead of hiding or refreshing values externally, we diversify

the value representation itself. To preserve the dominant numerical meaning, the rewrite avoids the sign bit, exponent field, and high- significance mantissa bits [21], modifying only the low-order mantissa bits of selected floating-point tensor values.

The selection is utility-aware rather than arbitrary. High-entropy intermediate tensors contain diverse values whose fine-grained differences often encode useful downstream signals. In contrast, low-entropy and repetition-dominant states concentrate around a small set of coarse numerical outcomes; their dominant inference meaning is already captured by the repeated value, leaving the low-order mantissa bits with comparatively little additional information. Rewriting these bits is therefore expected to be less disruptive than perturbing high-entropy states. This utility preference aligns with the leakage profile: the same low-entropy, repetition-dominant states create dense ciphertext collision patterns and thus require stronger protection.

From a systems perspective, this rewriting design is also attractive. Mask-based protection mechanisms [57] require additional mask state and explicit mask removal before subsequent computations can use the protected values. Memory-interleaving mechanisms require additional memory writes to ensure freshness for repeated stores [48]. In contrast, value rewriting changes the machine-visible representation within the protected tensor write itself without inserting an explicit demasking step. Once materialized in the tensor, the rewritten values become part of the numerical state consumed by later operators. The injected diversification can therefore propagate through downstream tensor computations via ordinary data dependencies, rather than being reintroduced by a separate protection pass at each later store.

Local Rewrite Rule and State Evolution In our realization, the rewrite operation is applied to protected tensor writes, not to the arithmetic operators themselves. For a protected write event (a, t), the a identifies the protected write site, and t denotes the t-th update to that site during execution. Let $y_{a,t}$ be the floating-point value produced by the original computation before it is materialized in memory. We denote its ordinary B-bit packed machine representation by $M_{a,t}^{\text{orig}} \triangleq \text{pack}(y_{a,t}) \in \{0,1\}^B$.

The value actually stored by a protected write operation is obtained by replacing only a compact low-order window of this representation:

$$M_{a,t}^{\text{new}} = \pi_H(M_{a,t}^{\text{orig}}) \parallel \sigma \parallel \psi_{a,t}^{\text{new}}. \tag{22}$$

$\pi_H(M_{a,t}^{\text{orig}})$ is the preserved high-order region of the original packed representation; $\sigma \in \{0,1\}^f$ is a fixed signature, which maintains the scale of the perturbation scale and $\psi_{a,t}^{\text{new}} \in \{0,1\}^p$ is the payload committed at the current update to ensure freshness of the resulting value.

The payload evolves from the old local state when the point is already in the defended regime and is otherwise initialized from the current run seed:

$$\psi_{a,t}^{\text{new}} = \begin{cases} \left(\pi_P(M_{a,t}^{\text{old}}) + \Delta\right) \bmod 2^p, & \text{if } \pi_F(M_{a,t}^{\text{old}}) = \sigma, \\ \text{Seed}_a, & \text{otherwise.} \end{cases} \tag{23}$$

π_F extracts the signature region of the protected window, π_P extracts the payload region, and Δ is the payload update step, determining how the protected state moves within the payload space after a point enters a protected state. A non-zero step size already avoids immediate self-repetition, but a step size coprime to 2^p is better because it traverses the entire payload space before looping.

The purpose of Seed_a is to provide an initial counter for each protected point. For security, the seed value for each address should be different. Furthermore, to minimize the use of the extra memory, we set a global Seed for one inference, from which the Seed_a for different addresses is derived. During each inference run, the Seed is refreshed, so the initial protection state of one run is independent of the initial protection state of other runs.

5.3 Security Analysis

Static-value Collision Reduction After the defense, we define the conditional distribution is

$$q(u \mid m) = \Pr[M_{a,t}^{\text{new}} = u \mid M_{a,t}^{\text{orig}} = m]. \tag{24}$$

If defended families associated with different original value m do not significantly overlap, the defended collision probability becomes

$$\widetilde{\text{CP}}_a \approx \sum_m [\Pr[M_{a,t}^{\text{orig}} = M_{a,t'}^{\text{orig}}] \sum_u q(u \mid m)^2]. \tag{25}$$

This expression shows that concentrated collision mass is redistributed over a larger defended family.

Transition Collision Reduction The same rewrite also reduces the recurrence of downstream transition collisions through propagation from the upstream protected point. Let $\widetilde{V}_{u,t-1} = \mathcal{R}_u(M_{u,t-1}^{\text{new}})$ denote the corresponding defended upstream value used by the downstream computation. If the downstream increment is generated by an operator-specific map $g_a(\cdot)$, then the defended increment is $\widetilde{\Delta}_{a,t} = g_a(\widetilde{V}_{u,t-1})$. For a fixed pre-update state y, define

$$\alpha_{a,t}(y) = \Pr\left[\text{pack}(y + \widetilde{\Delta}_{a,t}) = \text{pack}(y)\right]. \tag{26}$$

Then the defended transition collision probability becomes

$$\widetilde{\text{TCP}}_a = \mathbb{E}[\alpha_{a,t}(Y_{a,t-1})]. \tag{27}$$

Residual Leakage and Intrinsic Floors The method is intentionally selective rather than universal. It operates only on designated protected points. Residual leakage may arise from untargeted static repetitions, or the defense-induced changes that propagate downstream but do not yield observable differences in the resulting packed representations. Another class of residual collisions is largely determined when the model is compiled into an executable form and therefore tends to exhibit relatively stable patterns across repeated executions of the same model. These collisions primarily reflect implementation decisions made by the specific compiler, rather than victim-specific information. These effects create an intrinsic residual floor. The security objective, however, is not to eliminate all ciphertext side-channel leakage, but to suppress dominant victim-specific regularities, thereby limiting the extent to which the remaining observations improve victim-specific inference beyond the attacker's prior knowledge.

6 Implementation and Experimental Setup

Platform and Protocol. Our evaluation was conducted on an AMD EPYC 9474F server running Ubuntu 24.04 with Linux kernel 6.11. The software stack comprises Python 3.10.19, GCC 13.3, and

Clang/LLVM 14.0. While the current prototype is implemented within the LLVM-based toolchain, the underlying design does not depend on LLVM-specific features and is therefore portable to other compiler frameworks. All reported execution times represent CPU-side measurements obtained on this platform.

We employ two representative deep learning compiler backends: TVM and Glow.

- Within TVM, we evaluate both the native VM runtime and the Ahead-of-Time (AOT) runtime, capturing performance profiles for both a VM-based execution path and a statically exported deployment path.
- Within Glow, we consider both the generic `image-classifier` path via the Interpreter backend and the `modelw-compiler` path, which emits a standalone CPU bundle runner.

We abbreviate the evaluated compiler/runtime paths as follows: **TV** denotes TVM native VM runtime, **TA** denotes TVM AOT runtime, **IC** denotes Glow's `image-classifier` path, and **MC** denotes Glow's `model-compiler` path. To ensure a controlled comparison, all exogenous factors (including model weights, dataset splits, and inference hyperparameters) remain invariant within each backend. This experimental control ensures that observed performance deltas are strictly attributable to the defense mechanism rather than artifacts of the compilation pipeline or data variance.

Implementation. The implementation realizes the rewrite as a backend writeback transformation rather than as a source-level operator change. Protection writeback sites are identified after the model compiler's Backend IR optimization has completed. These sites may appear either as native backend instructions, or as write-backs that graph-level fusion has already absorbed into a fused operator. The former is identified via structural matching, and the latter leverages the backend fusion-metadata field set by the fused instruction at the model graph-level IR stage and later passed to the fused kernel as the `actType` parameter. For example, a lowered ReLU is identified by matching the canonical $\max(x, 0)$ pattern, and a fused convolution and ReLU operator is identified by its metadata field indicating `activation=RELU`. Then, after architecture-independent optimizations in LLVM, the instrumentation rewrites the value committed at each marked writeback. The rewrite sequence reads the newly computed operator result and the previous value at the destination element, updates the low-order signature-payload window, and forms the combined representation as the value to be committed. This organization localizes the transformation to selected writebacks, modifying only the bit-level representation of the value written to memory while preserving the operator schedule, loop nest, tensor layout, and destination-address computation.

Ciphertext Tracing and Acquisition. Ciphertext side-channel observation follows the standard CIPHERLEAKS [42] model. This model records the last ciphertext written within a memory page and checks for page-level ciphertext collisions. Notably, this model does not inherently reveal the underlying plaintext, the specific execution phase, or the semantic state transitions. To rigorously analyze the provenance of these transitions, we employ Intel Pin 4.1 [45] for instruction-level instrumentation.

By collecting auxiliary dynamic write traces, we simulate a maximum-capability adversary. We further integrate dynamic taint analysis [36, 59] to isolate write operations dependent on model inputs, thereby filtering out system-level memory activity unrelated to the private data. This instrumentation-based approach represents a more robust theoretical adversary than existing practical attacks which may rely on coarser binary transformation sequences, by granting visibility into all memory write events.

Table 2: Datasets and model coverage in the evaluation.

Dataset	Domain	Model–ID pairs
MNIST [17]	Digit Recognition	LeNet [38] (**T01**), VGGNet [54] (**T02**), ResNet [23] (**T06**), MobileNet [24] (**T11**)
CIFAR [37]	Natural-Image Classification	VGGNet (**T03**), SqueezeNet [28] (**T04**), ResNet (**T07**), MobileNet (**T12**)
ImageNet [16]	Natural-Image Classification	SqueezeNet (**T05**), ResNet (**T08**), DenseNet [27] (**T15**)
CelebA [44]	Face Identity	ResNet (**T09**), MobileNet (**T13**), DenseNet (**T16**)
Chest X-ray [56]	Medical Image Analysis	ResNet (**T10**), MobileNet (**T14**), DenseNet (**T17**)

Datasets and Models Used in the Evaluation. Table 2 details the NN architectures and datasets employed in our evaluation. We selected a diverse suite of models to ensure structural variety, spanning standard convolutional networks to more complex topologies. The associated datasets encompass a broad spectrum of real-world applications, including handwritten digit recognition, natural image classification, biometric face recognition, and medical diagnostics. For consistency and clarity, we utilize the unique identifiers established in Table 2 throughout the remainder of this paper.

7 Evaluation

We first evaluate collision reduction together with task utility and select the perturbation scale for the remaining experiments (Sec. 7.1). At the selected scale, we then evaluate the effectiveness of the proposed defense against two known attacks (Sec. 7.2), examine whether the defense generalizes across backends (Sec. 7.3), and report the task utility and runtime overhead of our proposed defense compared with existing ciphertext side-channel mitigation techniques (Sec. 7.4). Throughout this section, Def^{-} and Def^{+} denote experiments without and with defense, respectively.

7.1 Collision Evidence Analysis

Before conducting the collision evidence analysis, we first present how the perturbation scale is selected. The proposed defense mechanism incorporates an adjustable perturbation intensity, providing a tunable control knob to manage the trade-off between collision evidence leakage suppression and task utility.

Note that changing the perturbation scale has quite limited effect on the reduced number of static-value collision evidence once the protected static values are diversified. Its main effect is on transition collision evidence: a perturbation with a magnitude below $\frac{1}{2}\text{ulp}(x)$ may not result in a representable change in floating point arithmetic [21], failing to alter the resulting trace, whereas an excessively large one can distort intermediate activations, leading to a degradation of task utility. Detailed analysis is as follows.

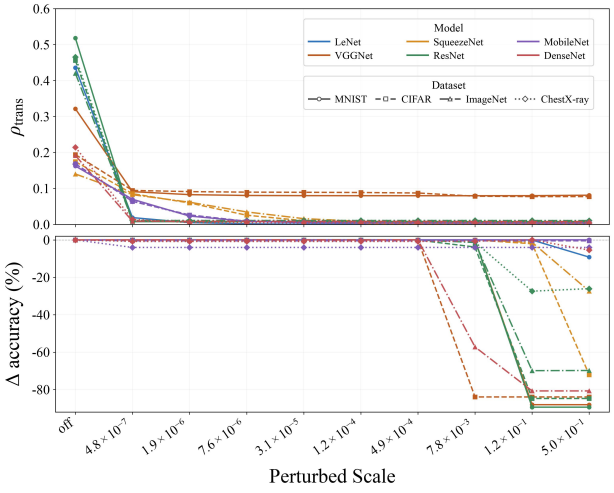


Figure 3: Trade-off between ρ_{trans} and task utility under varying perturbation scales.

Static-Value Collision Evidence. We follow Eq. (17) and measure it over the writes at protected ReLU outputs, where the static-value collision probability is dominated by p_0. Without defense, we measure the ReLU outputs over the total execution interval containing multiple inferences and calculate the values of ρ_{static}, ranging from 40% to 63% across workloads. After rewriting these protected values, we find that they become more dispersed with no pronounced concentration, and the previously prominent static value 0 is no longer observed, indicating that ρ_{static} reduces to *zero*.

Transition Collision Evidence. The transition collision evidence in Fig. 3 show that the undefended ρ_{trans} ranges from approximately 14% to 52% across workloads. Enabling the defense substantially reduces ρ_{trans}, and the rate decreases further as the perturbation scale increases. At scales on the order of 10^{-4}, the results for most non-VGGNet workloads are at or below approximately 0.85%. These results demonstrate the effectiveness of the defense, and confirm that transition collision evidence depends on whether the perturbation remains distinguishable in subsequent writes. Within this overall trend, the two VGGNet workloads retain noticeably higher values than the other workloads, reaching 8.12% on MNIST and 7.73% on CIFAR when the perturbation scale exceeds 10^{-5}. Further analysis shows that most of the remaining collisions (92.73% and 97.54% of the respective reported ρ_{trans} values) arise from the compilation-determined residual collision evidence described in Sec. 5.3, which does not provide victim-specific information. After excluding this component, the remaining potentially exploitable collision evidence decreases to 0.58% on MNIST, and 0.19% on CIFAR, respectively.

Parameter Selection. Fig. 3 also shows the trade-off between ρ_{trans} and task utility as the perturbation scale varies, with a range in which ρ_{trans} is low while task utility remains high (with degradation kept within approximately 4%), suggesting the viability of our defense. In the subsequent evaluations, we set the default perturbation scale at 4.9×10^{-4}, providing a strong perturbation without compromising model utility.

Table 3: Trace-to-image reconstruction results across datasets, including results for h and x^*.

Dataset	Ref. Acc.	Setting	Recon. Acc.		$\text{TU}_\tau^{\text{cons}}(A,K)$		$\text{TU}_\tau^{\text{sim}}(A,K)$		MSE	
			h	x^*	h	x^*	h	x^*	h	x^*
CIFAR	0.7860	Def^-	0.5340	0.4040	0.5100	0.3231	0.1584	0.1189	0.0578	0.0950
		Def^+	0.1080	0.1020	0.0000	0.0000	0.0000	-0.0009	0.2013	0.2450
ImageNet	0.8350	Def^-	0.5015	0.3430	0.4866	0.3208	0.4308	0.0779	0.0394	0.1703
		Def^+	0.1265	0.0520	0.1000	0.0284	0.2225	0.0340	0.1008	0.2251
MNIST	0.9820	Def^-	0.8620	0.6880	0.7619	0.5960	0.1899	0.1318	0.0803	0.1937
		Def^+	0.1900	0.1000	0.0881	0.0032	0.0058	0.0002	0.5128	0.3876
Chest X-ray	0.7770	Def^-	0.6647	0.5317	0.1267	0.0439	0.2712	0.1065	0.0116	0.0697
		Def^+	0.5016	0.4522	0.0002	-0.0173	0.0031	-0.0039	0.1369	0.1919

Table 4: Trace-to-label prediction results across datasets. For ImageNet, the three Def^- and Def^+ variants are evaluated on ciphertext traces obtained from `transition1.conv`, `transition2.conv`, and `transition3.conv`.

Dataset	Ref. Acc.	Setting	Pred. Acc.	$\text{TU}_\tau^{\text{cons}}(A,K)$
CIFAR	0.7860	Def^-	0.7564	0.7652
		Def^+	0.1002	0.0001
ImageNet	0.8350	Def^-: `transition1.conv`	0.4368	0.4248
		Def^-: `transition2.conv`	0.6160	0.6012
		Def^-: `transition3.conv`	0.7484	0.7508
		Def^+: `transition1.conv`	0.2400	0.2244
		Def^+: `transition2.conv`	0.1656	0.1460
		Def^+: `transition3.conv`	0.0200	0.0000
MNIST	0.9820	Def^-	0.9540	0.8360
		Def^+	0.0980	0.0000
Chest X-ray	0.7770	Def^-	0.7087	0.1915
		Def^+	0.5128	0.0000

7.2 Effectiveness against Known Attacks

We evaluate the effectiveness of the defense against known attacks using the following attack-specific recovery objectives:

- *Trace-to-image recovery*, as in CipherSteal, evaluates semantic recovery and, when reconstructed images are available, additionally evaluates pixel-level recovery.
- *Trace-to-label prediction* measures semantic recovery by predicting the target label directly from the trace, as described in Sec. 3.2.

Trace-to-Image Recovery. We evaluate trace-to-image recovery from both metric-based and reconstruction-level perspectives. For the metric-based evaluation, we instantiate s_τ in Eq. (14) with semantic prediction consistency and image similarity, yielding $\text{TU}_\tau^{\text{cons}}(A,K)$ and $\text{TU}_\tau^{\text{sim}}(A,K)$, respectively. We additionally report MSE to measure pixel-space reconstruction error. Table 3 and Fig. 4 jointly present the trace-to-image recovery results. The table quantifies the attacker's recovery advantage for the trace-derived sketch h and the prior-refined reconstruction x^*, while the figure visualizes the corresponding recovery outputs before and after defense.

As shown in Table 3, enabling the defense reduces both TU_τ scores across all datasets and increases the corresponding MSE, indicating that the defended trace provides substantially less additional semantic and pixel-level information. Fig. 4 further shows that the defense affects the two recovery stages differently. After defense, the sketch h largely loses victim-specific structure and becomes visually similar across different victim inputs. In contrast,



(a) CIFAR (b) ImageNet (c) MNIST (d) Chest X-ray

Figure 4: Trace-to-image recovery results of the CipherSteal attack before and after defense. Within each five-column sample group, columns from left to right show the original input x, the undefended sketch h, the undefended reconstruction x^*, the defended sketch h, and the defended reconstruction x^*.

the defended reconstruction x^* may remain visually plausible because the generator continues to synthesize details from its learned prior. However, these details no longer align with the corresponding victim inputs and frequently reflect unrelated or repeated dataset-typical content. These results show that the defense suppresses the victim-specific evidence preserved in h, while the visual content retained in x^* is primarily supplied by the attacker's prior.

Trace-to-Label Prediction. We next examine the label information that can be inferred directly from ciphertext traces without reconstructing an image. Following Sec. 3.2, the trace-based inference model F_c maps each ciphertext trace to a label prediction. Using prediction consistency as s_τ, we instantiate the previously defined $\mathrm{TU}_\tau(A, K)$ metric as $\mathrm{TU}_\tau^{\mathrm{cons}}(A, K)$ and report the corresponding results in Table 4. The comparison between Def$^-$ and Def$^+$ shows that the defense substantially reduces the label information recoverable from ciphertext traces.

7.3 Generalization across Compiler/Runtime Paths

The collision analysis in Sec. 7.1 is based on the **MC** path. To evaluate generalization of the proposed defense, we apply the selected perturbation scale to all four compiler/runtime paths. Static-value collisions are governed primarily by the output distribution and therefore vary little across backends. Transition collisions, in contrast, depend on how a backend materializes intermediate writes (*i.e.* whether accumulations are updated in place, how buffers are reused, and whether fused operators keep intermediate results in registers). We therefore report ρ_{trans} in Table 5. Overall, the defense broadly reduces transition collisions. Under the Glow framework, ρ_{trans} falls below 1% in the majority of workloads. Substantial reductions are also achieved for the TVM framework, where the residual collisions are dominated by structural writes arising from the trailing buffer regions (zeroed during buffer initialization) left untouched by tensor computation but preserved through bulk memory copies.

7.4 Utility, Overhead, and Comparative Positioning

Task Utility. In Table 6, the Def$^+$ scores remain close to the corresponding Def$^-$ reference, indicating that the selected operating point introduces only limited inference perturbation in practice. These deviations are small and mixed in sign, consistent with the fact that the defense perturbs the protected intermediate values and therefore perturbs downstream logits, rather than enforcing exact preservation of every final prediction. As a result, samples near

Table 5: ρ_{trans} before and after defense (Def$^-$ vs. Def$^+$) across all combinations of models, datasets, and frameworks.

Task	MC		IC		TV		TA	
	Def$^-$	Def$^+$	Def$^-$	Def$^+$	Def$^-$	Def$^+$	Def$^-$	Def$^+$
T01	43.56%	0.04%	39.03%	0.58%	5.66%	2.79%	11.78%	0.78%
T02	32.15%	7.97%	14.56%	0.32%	4.19%	1.04%	14.13%	1.18%
T03	19.41%	8.73%	4.25%	0.99%	10.14%	2.73%	8.15%	1.97%
T04	17.45%	0.12%	9.43%	0.60%	8.75%	2.12%	15.65%	2.43%
T05	14.02%	0.58%	4.50%	0.11%	7.59%	2.47%	14.75%	2.78%
T06	51.83%	0.75%	0.80%	0.06%	12.47%	3.07%	9.30%	3.31%
T07	45.61%	0.80%	0.79%	0.08%	12.94%	3.06%	9.50%	3.35%
T08	42.01%	1.07%	0.88%	0.11%	9.53%	1.94%	10.25%	2.15%
T09	47.38%	1.39%	0.46%	0.05%	9.84%	1.90%	11.62%	2.55%
T10	46.56%	1.09%	0.42%	0.06%	11.14%	2.04%	12.32%	2.60%
T11	16.23%	0.38%	1.27%	0.08%	14.40%	2.96%	17.00%	3.42%
T12	16.84%	0.38%	1.54%	0.12%	15.14%	2.99%	17.37%	3.38%
T13	17.20%	0.37%	2.05%	0.04%	12.25%	1.84%	16.02%	2.04%
T14	16.81%	0.35%	2.08%	0.05%	12.41%	1.90%	16.01%	2.07%

Table 6: Task utility comparison results (with absolute changes from Def$^-$ in parentheses).

Task	Def$^-$	MC (Def$^+$)	IC (Def$^+$)	TV (Def$^+$)	TA (Def$^+$)
T01	98.91%	98.91%(+0.0000)	98.91%(+0.0000)	98.91%(+0.0000)	98.91%(+0.0000)
T02	99.47%	99.47%(+0.0000)	99.47%(+0.0000)	99.47%(+0.0000)	99.47%(+0.0000)
T03	93.98%	93.89%(-0.0009)	93.94%(-0.0004)	93.85%(-0.0013)	93.85%(-0.0013)
T04	92.17%	92.08%(-0.0009)	92.19%(+0.0002)	92.19%(+0.0002)	92.20%(+0.0003)
T05	63.12%	63.12%(+0.0000)	63.12%(+0.0000)	63.08%(-0.0004)	63.12%(+0.0000)
T06	99.54%	99.55%(+0.0001)	99.54%(+0.0000)	99.55%(+0.0001)	99.55%(+0.0001)
T07	94.80%	94.84%(+0.0004)	94.81%(+0.0001)	94.76%(-0.0004)	94.75%(-0.0005)
T08	71.84%	71.80%(-0.0004)	71.84%(+0.0000)	71.92%(+0.0008)	71.80%(-0.0004)
T09	53.32%	53.18%(-0.0014)	53.42%(+0.0010)	53.02%(-0.0030)	52.98%(-0.0034)
T10	77.67%	77.35%(-0.0032)	77.67%(+0.0000)	77.68%(+0.0001)	77.68%(+0.0001)
T11	99.45%	99.47%(+0.0002)	99.45%(+0.0000)	99.45%(+0.0000)	99.45%(+0.0000)
T12	93.98%	93.48%(-0.0050)	94.02%(+0.0004)	94.01%(+0.0003)	94.00%(+0.0002)
T13	59.14%	58.76%(-0.0038)	59.06%(-0.0008)	58.88%(-0.0026)	58.86%(-0.0028)
T14	77.42%	77.44%(+0.0002)	77.44%(+0.0002)	77.46%(+0.0003)	77.46%(+0.0003)
T15	82.76%	82.80%(+0.0004)	82.72%(-0.0004)	82.84%(+0.0008)	82.76%(+0.0000)
T16	73.36%	72.98%(-0.0038)	73.50%(+0.0014)	73.50%(+0.0014)	73.54%(+0.0018)
T17	52.17%	51.34%(-0.0083)	52.18%(+0.0001)	52.18%(+0.0001)	52.18%(+0.0001)

Table 7: Runtime overhead relative to Def$^-$. Each value reports the median over five de-noised runs.

Task	Model/Dataset	MC	IC	TV	TA
T01	LeNet/MNIST	+4.90%	+3.85%	+10.11%	+12.88%
T03	VGGNet/CIFAR	+26.89%	+0.56%	+0.63%	+0.51%
T07	ResNet/CIFAR	+8.50%	+0.74%	+0.67%	+0.64%
T10	ResNet/Chest X-ray	+6.52%	+0.29%	+0.72%	+0.89%
T17	DenseNet/Chest X-ray	+53.96%	+2.87%	+2.87%	+2.81%

Table 8: Comparison of task utility and inference runtime overhead of different defense schemes. Results report test outcomes on selected task groups for three defense schemes (Ours, CipherFix-style, and Zebrafix-style).

Task	Task Utility				Time Overhead		
	Ref. (Def$^-$)	Ours (Def$^+$)	CipherFix-style	Zebrafix-style	Ours (Def$^+$)	CipherFix-style	Zebrafix-style
T01	0.9891	0.9891	0.9891	0.9891	+4.68%	+962.65%	+588.19%
T03	0.9398	0.9397	0.9398	0.9398	+14.69%	+2466.30%	+1943.12%
T07	0.9409	0.9414	0.9409	0.9409	+5.90%	+691.96%	+658.96%

decision boundaries may move in either direction, producing small aggregate gains or losses without indicating systematic degradation. Taking $\epsilon = 0.01$ as a conservative application-level budget, we find that all observed utility losses in Table 6 remain below this threshold.

Runtime Overhead. As Table 7 shows, the overhead variation is closely related to the model architecture and backend-specific implementation choices. Our defense mechanism is applied only to a selected set of protected points $\mathcal{M}_{\mathcal{P}}$, rather than uniformly to every operator. As discussed in Sec. 4.3, architectures with more ReLU-like layers tend to increase the size of $\mathcal{M}_{\mathcal{P}}$ and invoke the defense mechanism more frequently, making the relative overhead more pronounced. For example, VGGNet and DenseNet use relatively more ReLU activation functions, resulting in overhead increases of 26.89% and 53.96%, respectively.

Comparative Organization. We compare our defense with CipherFix and Zebrafix in NN inference, under the same model configuration and inference flow. As shown in Table 8, all methods achieve task utility comparable to that without defense, with ours being the only one that exhibits a slight deviation. On the other hand, CipherFix and Zebrafix introduce additional work before computation by avoiding perturbation of intermediate values through maintaining a protected memory representation and removing or processing auxiliary mask/counter states. Our method performs smaller bounded numerical perturbations at selected points in exchange for lower runtime overhead.

8 Related Work

Ciphertext Side Channels in TEEs. Ciphertext side channels arise in TEEs that use deterministic memory encryption at fixed physical addresses. CIPHERLEAKS [41, 42] showed that such leakage can break constant-time cryptographic implementations in AMD SEV-SNP, and follow-up work further expanded the attack patterns and targets [52]. This vulnerability is not limited to AMD SEV-SNP: Hygon CSV3 exhibits similar deterministic-encryption weaknesses, enabling ciphertext block replaceability attacks within cache lines [49]. Recent hardware-level attacks further show that ciphertext observability can extend beyond software-visible traces. WireTap [53] demonstrated passive DDR4-bus sniffing against Intel Scalable SGX, while BatteringRAM [15] extended the threat to active manipulation. Even with *ciphertext hiding* in AMD SEV-SNP [3, 4] or memory integrity in Intel TDX, DDR5-bus observations may still expose ciphertext patterns and sensitive encrypted data [11].

Ciphertext Side Channels for NNs. NN workloads expose a large value-based leakage surface under TEEs. Intermediate activations, partial sums, and normalization statistics are repeatedly materialized in memory, and their value distributions often reflect both model structure and input-dependent behavior. Recent attacks such as CipherSteal [62] and HyperTheft [61] showed that ciphertext side channels can be weaponized against NN inference to recover sensitive information. Relocate-Vote [60] further highlighted the role of sparsity by exploiting prevalent zero values and their stable ciphertext patterns to infer activation states.

Detection and Mitigation. Existing work has proposed both analysis tools and software mitigations for ciphertext side channels. CipherH [18] combines whole-program taint tracking with function-level reasoning to identify program points that induce ciphertext leakage, while SNPeek [63] provides a toolkit for collecting ciphertext traces and evaluating leakage in confidential VMs. These tools support vulnerability discovery and measurement, but do not by themselves provide low-overhead protection for tensor-intensive NN inference. Software defenses such as CipherFix [57], Zebrafix [48], and CipherGuard [31] aim to break ciphertext determinism by protecting memory stores. CipherFix uses secret tracking and binary instrumentation to mask protected stores with fresh randomness, whereas Zebrafix uses compiler-based interleaving and counters to provide deterministic freshness, at the cost of additional memory traffic and reported 50% memory overhead. These approaches are general and effective for their target settings, but they are primarily designed around store-level protection for conventional security-sensitive code. Directly applying them to NNs is challenging because inference involves dense tensor write-backs, large-scale dataflow, and operator-level value structures that differ from cryptographic workloads.

9 Conclusion

This paper revisits ciphertext side-channel attacks against NN inference in TEEs through the lens of what the ciphertext trace actually contributes beyond the attacker's offline prior. By separating prior-driven plausibility from victim-specific trace utility, we identify static-value collisions and transition collisions as the main execution-level leakage sources exposed by deterministic memory encryption. Based on this observation, we design a selective perturbation-based defense, rather than protecting every write related to secret. Our evaluation shows that a properly calibrated perturbation scale can preserve model utility while substantially reducing collision evidence and preventing CipherSteal-style reconstruction from succeeding. These results suggest that workload-aware collision reduction can provide a more practical performance-security trade-off for NN inference than generic hardening for all tainted writes.

Acknowledgments

The authors from Shanghai Jiao Tong University were partially supported by the National Natural Science Foundation of China under Grant No. 62472281. Yinqian Zhang was partially supported by the National Natural Science Foundation of China under Grant No. 62361166633 and Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under Grant No. JYB2025XDXM114. We thank our anonymous reviewers for their insightful comments, which have significantly improved the paper.

References

[1] Advanced Micro Devices, Inc. 2022. *Ciphertext Side Channels on AMD SEV*. AMD Security Bulletin AMD-SB-1033. https://www.amd.com/en/resources/product-security/bulletin/amd-sb-1033.html

[2] Advanced Micro Devices, Inc. 2025. *SEV Ciphertext Side Channel Attacks*. AMD Security Brief AMD-SB-3021. https://www.amd.com/en/resources/product-security/bulletin/amd-sb-3021.html

[3] Advanced Micro Devices, Inc. 2026. *AMD Secure Encrypted Virtualization (SEV)*. https://www.amd.com/en/developer/sev.html

[4] Advanced Micro Devices, Inc. 2026. *SEV Secure Nested Paging Firmware ABI Specification*. Publication No. 56860, Revision 1.59 PUB. https://docs.amd.com/v/u/en-US/56860_PUB_SEV_SNP

[5] Dhruv Deepak Agarwal and Aswani Kumar Cherukuri. 2025. Confidential Computing for Cloud Security: Exploring Hardware based Encryption Using Trusted Execution Environments. *arXiv preprint arXiv:2511.04550* (2025).

[6] Arm Limited. 2025. *Arm Confidential Compute Architecture Software Stack*. Document DEN0127, Version 3.0, Issue 06. https://developer.arm.com/documentation/den0127/latest

[7] Mohammad Fakhruddin Babar and Monowar Hasan. 2023. Trusted deep neural execution—a survey. *IEEE Access* 11 (2023), 45736–45748.

[8] Keke Chen. 2023. Confidential high-performance computing in the public cloud. *IEEE Internet Computing* 27, 1 (2023), 24–32.

[9] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 578–594.

[10] Marcin Chrapek, Anjo Vahldiek-Oberwagner, Marcin Spoczynski, Scott Constable, Mona Vij, and Torsten Hoefler. 2024. Fortify your foundations: Practical privacy and security for foundation model deployments in the cloud. *arXiv preprint arXiv:2410.05930* (2024).

[11] Jalen Chuang, Alex Seto, Nicolas Berrios, Stephan Van Schaik, Christina Garman, and Daniel Genkin. 2026. Tee. fail: Breaking trusted execution environments via ddr5 memory bus interposition. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4408–4426.

[12] Thomas M. Cover and Joy A. Thomas. 2006. *Elements of Information Theory* (2 ed.). Wiley-Interscience. doi:10.1002/047174882X

[13] Imre Csiszár and János Körner. 2011. *Information theory: coding theorems for discrete memoryless systems*. Cambridge University Press.

[14] Scott Cyphers, Arjun K Bansal, Anahita Bhiwandiwalla, Jayaram Bobba, Matthew Brookhart, Avijit Chakraborty, Will Constable, Christian Convey, Leona Cook, Omar Kanawi, et al. 2018. Intel ngraph: An intermediate representation, compiler, and executor for deep learning. *arXiv preprint arXiv:1801.08058* (2018).

[15] Jesse De Meulemeester, David Oswald, Ingrid Verbauwhede, and Jo Van Bulck. 2026. Battering RAM: Low-cost interposer attacks on confidential computing via dynamic memory aliasing. In *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4391–4407.

[16] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 248–255.

[17] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine* 29, 6 (2012), 141–142.

[18] Sen Deng, Mengyuan Li, Yining Tang, Shuai Wang, Shoumeng Yan, and Yinqian Zhang. 2023. CipherH: Automated Detection of Ciphertext Side-channel Vulnerabilities in Cryptographic Implementations. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6843–6860.

[19] Morris J. Dworkin. 2010. *Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Storage Devices*. NIST Special Publication 800-38E. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-38E

[20] Anila Gogineni. 2025. Confidential Computing Architectures for Enhanced Data Security in Cloud Environments. *International Journal on Science and Technology* 16, 2 (2025). doi:10.71097/IJSAT.v16.i2.3172

[21] David Goldberg. 1991. What every computer scientist should know about floating-point arithmetic. *ACM computing surveys (CSUR)* 23, 1 (1991), 5–48.

[22] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *International conference on machine learning*. PMLR, 1737–1746.

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[24] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017).

[25] Bin Hu, Yan Wang, Jerry Cheng, Tianming Zhao, Yucheng Xie, Xiaonan Guo, and Yingying Chen. 2023. Secure and efficient mobile DNN using trusted execution environments. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*. 274–285.

[26] Weizhe Hua, Zhiru Zhang, and G Edward Suh. 2022. Reverse-Engineering CNN Models Using Side-Channel Attacks. *IEEE Des. Test* 39, 4 (2022), 15–22.

[27] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4700–4708.

[28] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size. *arXiv preprint arXiv:1602.07360* (2016).

[29] Intel Corporation. 2021. *3rd Gen Intel Xeon Scalable Processors for IoT*. Product Brief. https://www.intel.com/content/dam/www/central-libraries/us/en/documents/3rd-gen-xeon-sp-for-iot-product-brief.pdf

[30] Intel Corporation. 2025. *Intel Trust Domain Extensions (Intel TDX)*. White Paper. Intel Corporation.

[31] Ke Jiang, Sen Deng, Yinshuai Li, Shuai Wang, Tianwei Zhang, and Yinqian Zhang. 2025. CipherGuard: Compiler-aided Mitigation against Ciphertext Side-channel Attacks. *arXiv preprint arXiv:2502.13401* (2025).

[32] Simon Johnson, Raghunandan Makaram, Amy Santoni, and Vinnie Scarlata. 2021. *Supporting Intel SGX on Multisocket Platforms*. White Paper. Intel Corporation. https://www.intel.com/content/www/us/en/content-details/843058/supporting-intel-sgx-on-multisocket-platforms.html

[33] David Kaplan. 2017. *Protecting VM Register State with SEV-ES*. White Paper 70364. Advanced Micro Devices. https://docs.amd.com/v/u/en-US/Protecting-VM-Register-State-with-SEV-ES

[34] David Kaplan, Jeremy Powell, and Tom Woller. 2020. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. White Paper 70366. Advanced Micro Devices. https://docs.amd.com/v/u/en-US/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more

[35] David Kaplan, Jeremy Powell, and Tom Woller. 2021. *AMD Memory Encryption White Paper*. White Paper 70363. Advanced Micro Devices. https://docs.amd.com/v/u/en-US/memory-encryption-white-paper

[36] Jacob Kreindl, Daniele Bonetta, Lukas Stadler, David Leopoldseder, and Hanspeter Mössenböck. 2021. Low-overhead multi-language dynamic taint analysis on managed runtimes through speculative optimization. In *Proceedings of the 18th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes*. 70–87.

[37] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning Multiple Layers of Features from Tiny Images*. Technical Report. University of Toronto, Toronto, Ontario, Canada.

[38] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.

[39] Taegyeong Lee, Zhiqi Lin, Saumay Pushp, Caihua Li, Yunxin Liu, Youngki Lee, Fengyuan Xu, Chenren Xu, Lintao Zhang, and Junehwa Song. 2019. Occlumency: Privacy-preserving remote deep-learning inference using SGX. In *The 25th Annual international conference on mobile computing and networking*. 1–17.

[40] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. 2021. The Deep Learning Compiler: A Comprehensive Survey. *IEEE Transactions on Parallel and Distributed Systems* 32, 3 (2021), 708–727. doi:10.1109/TPDS.2020.3030548

[41] Mengyuan Li, Luca Wilke, Jan Wichelmann, Thomas Eisenbarth, Radu Teodorescu, and Yinqian Zhang. 2022. A systematic look at ciphertext side channels on AMD SEV-SNP. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 337–351.

[42] Mengyuan Li, Yinqian Zhang, Huibo Wang, Kang Li, and Yueqiang Cheng. 2021. CIPHERLEAKS: Breaking constant-time cryptography on AMD SEV via the ciphertext side channel. In *30th USENIX Security Symposium (USENIX Security 21)*. 717–732.

[43] Yuepeng Li, Deze Zeng, Lin Gu, Quan Chen, Song Guo, Albert Zomaya, and Minyi Guo. 2021. Lasagna: Accelerating secure deep learning inference in sgx-enabled edge cloud. In *Proceedings of the ACM symposium on cloud computing*. 533–545.

[44] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. 2015. Deep learning face attributes in the wild. In *Proceedings of the IEEE international conference on*

computer vision. 3730–3738.

[45] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) *(PLDI '05)*. Association for Computing Machinery, New York, NY, USA, 190–200. doi:10.1145/1065010.1065034

[46] Zoltán Ádám Mann, Christian Weinert, Daphnee Chabal, and Joppe W. Bos. 2023. Towards Practical Secure Neural Network Inference: The Journey So Far and the Road Ahead. *ACM Comput. Surv.* 56, 5, Article 117 (2023), 37 pages. doi:10.1145/3628446

[47] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. 2024. Confidential vms explained: An empirical analysis of amd sev-snp and intel tdx. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8, 3 (2024), 1–42.

[48] Anna Pätschke, Jan Wichelmann, and Thomas Eisenbarth. 2025. ZebraFix: Mitigating Memory-Centric Side-Channel Leakage via Interleaving. In *2025 IEEE International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*. IEEE, 835–849. doi:10.1109/RAID67961.2025.00063

[49] Wei Peng, Yinshuai Li, and Yinqian Zhang. 2025. Shadows in Cipher Spaces: Exploiting Tweak Repetition in Hardware Memory Encryption. In *34th USENIX Security Symposium (USENIX Security 25)*. 5759–5776.

[50] Yury Polyanskiy and Yihong Wu. 2017. Strong data-processing inequalities for channels and Bayesian networks. In *Convexity and Concentration*. Springer, 211–249.

[51] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Garret Catron, Summer Deng, Roman Dzhabarov, Nick Gibson, James Hegeman, Meghan Lele, Roman Levenstein, et al. 2018. Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907* (2018).

[52] Benedict Schlüter, Christoph Wech, and Shweta Shinde. 2025. Heracles: Chosen plaintext attack on amd sev-snp. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 3810–3824.

[53] Alex Seto, Oytun Kuday Duran, Samy Amer, Jalen Chuang, Stephan van Schaik, Daniel Genkin, and Christina Garman. 2025. Wiretap: Breaking server sgx via dram bus interposition. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 708–722.

[54] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations (ICLR)*.

[55] Jatinder Singh, Jennifer Cobbe, Do Le Quoc, and Zahra Tarkhani. 2021. Enclaves in the clouds: Legal considerations and broader implications. *Commun. ACM* 64, 5 (2021), 42–51.

[56] Xiaosong Wang, Yifan Peng, Le Lu, Zhiyong Lu, Mohammadhadi Bagheri, and Ronald M Summers. 2017. Chestx-ray8: Hospital-scale chest x-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2097–2106.

[57] Jan Wichelmann, Anna Pätschke, Luca Wilke, and Thomas Eisenbarth. 2023. Cipherfix: Mitigating Ciphertext Side-Channel Attacks in Software. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6789–6806.

[58] Benjamin Wu, Trishita Tiwari, G Edward Suh, and Aaron B Wagner. 2021. Guessing Outputs of Dynamically Pruned CNNs Using Memory Access Patterns. *IEEE Computer Architecture Letters* 20, 2 (2021), 98–101.

[59] Babak Yadegari and Saumya Debray. 2014. Bit-level taint analysis. In *2014 IEEE 14th International Working Conference on Source Code Analysis and Manipulation*. IEEE, 255–264.

[60] Yuqin Yan, Wei Huang, Ilya Grishchenko, Gururaj Saileshwar, Aastha Mehta, and David Lie. 2025. Relocate-Vote: Using Sparsity Information to Exploit Ciphertext Side-Channels. In *34th USENIX Security Symposium (USENIX Security 25)*. 5699–5717.

[61] Yuanyuan Yuan, Zhibo Liu, Sen Deng, Yanzuo Chen, Shuai Wang, Yinqian Zhang, and Zhendong Su. 2024. Hypertheft: Thieving model weights from tee-shielded neural networks via ciphertext side channels. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4346–4360.

[62] Yuanyuan Yuan, Zhibo Liu, Sen Deng, Yanzuo Chen, Shuai Wang, Yinqian Zhang, and Zhendong Su. 2025. Ciphersteal: Stealing input data from tee-shielded neural networks with ciphertext side channels. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4136–4154.

[63] Ruiyi Zhang, Albert Cheu, Adria Gascon, Daniel Moghimi, Phillipp Schoppmann, Michael Schwarz, and Octavian Suciu. 2026. SNPeek: Side-Channel Analysis for Privacy Applications on Confidential VMs. In *Network and Distributed System Security Symposium (NDSS)*.

[64] SM Zobaed and Mohsen Amini Salehi. 2025. Confidential Computing Across Edge-To-Cloud for Machine Learning: A Survey Study. *Software: Practice and Experience* 55, 5 (2025), 896–924.

A Open Science

We release the implementation of our method. It includes the implementation of the evaluated attack pipeline and the mismatch-tracking baseline; code for measuring static-value collisions and transition collisions; the perturbation implementation and configuration files for model settings, protected layers, and defense parameters; scripts for evaluating mitigation effectiveness; and scripts for measuring usability and runtime overhead. The artifact is available at https://github.com/muccci/nn-ciphertext-defense.

B Ethical Considerations

This paper studies ciphertext side-channel leakage and defenses for TEE-protected NN inference. The work is dual-use: reproducing representative attacks and analyzing transition collisions and no-change events can help quantify risk and validate mitigations, but may also help adversaries reason about useful trace features.

We minimize potential harm by restricting all experiments to authorized, researcher-controlled environments. The evaluation uses public benchmarks, synthetic data, or inputs, models, and traces generated for this study. It does not collect traces from third-party production services, bypass access controls, interact with uninvolved users, or expose private user data. Any future deployment-specific vulnerabilities will be responsibly disclosed before publication.

C Generative AI Usage

The authors employed generative AI platforms (including GPT, Gemini, Claude, and Cursor) to refine the manuscript's prose and assist in developing data visualization scripts. To ensure academic integrity, all AI-generated suggestions, including technical descriptions, code, underwent rigorous manual validation. This process involved cross-referencing content with the original experimental data and re-executing scripts to confirm the accuracy of all figures and findings.